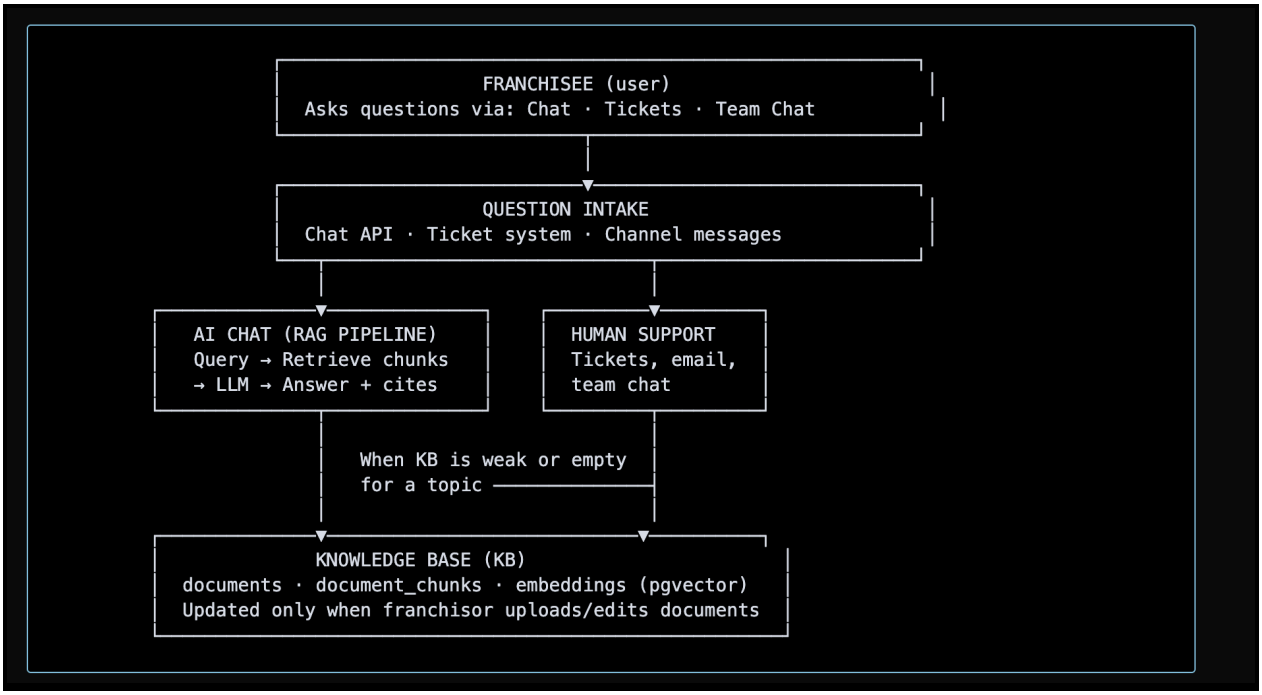
1. System context: the knowledge–answer loop

The platform serves **franchise brands**. Franchisees (location owners and staff) ask operational questions; the brand (franchisor) provides answers via documentation and support. The core "system" we care about is the **knowledge–answer loop**:

- **Knowledge base (KB):** Documents (manuals, SOPs, playbooks) that the brand uploads, stored and chunked for search.
- **Question channel:** Franchisees ask questions through an AI chat assistant (and optionally through support tickets, team chat, etc.).
- **Answer mechanism:** The AI uses a RAG pipeline (retrieve relevant chunks from the KB, then generate an answer with citations). If the KB has no or weak content on a topic, the AI cannot answer well.
- **Human support:** When the AI can't help, franchisees may open a support ticket or ask in team chat; franchisor staff answer manually.

So the system has two paths: **self-serve (AI + KB)** and **human support**. The quality and coverage of the KB directly determine how much load stays on self-serve versus support.



Today, **KB updates are decoupled from usage.** The franchisor adds or edits documents on their own schedule. There is no automatic signal that "your franchisees are asking about X and the KB doesn't cover it." So the system has a **one-way flow**: KB → RAG → answers. There is no **feedback loop** from "questions the system couldn't answer well" back into "what to add to the KB."

2. Leverage points

Leverage point = a place in the system where a small, targeted change produces a large improvement in outcomes (e.g. fewer repeat questions, less support load, better answers).

We have identified the following.

#	Leverage point	Where it lives	Why it's high leverage
L1	**Signal that the AI failed or was uncertain**	End of each chat response (orchestrator / post-processing)	Every "bad" or "I don't know" answer is a direct signal of a KB gap. Today this signal is either unused or only logged for analytics. Capturing it in a structured way (per question, with confidence) is the minimal change that makes the rest of the loop possible.
L2	**Structured storage of gaps**	New or extended table (`knowledge_gaps`) keyed by company + question/gap	Raw chat logs are too noisy. A deduplicated, company-scoped store of "questions we couldn't answer well" (with occurrence counts, timestamps, optional link to tickets) turns signal into **actionable list** for the franchisor. One place to look for "what to document next."
L3	**Prioritization of gaps**	Gap records + scoring (frequency, recency, # of users)	Not all gaps are equal. Ranking by impact (e.g. asked 3+ times, multiple locations) focuses human and AI effort on the gaps that will reduce the most support load and repeat questions. Small logic change, large effect on perceived value and adoption.

L4	**Reuse of the document pipeline for approved content**	Document ingestion (upload → chunk → embed)	The platform already has a pipeline that turns a document into searchable chunks and embeddings. If the Knowledge Builder produces **drafts** that are just "documents to be ingested," we don't build a second pipeline. We only add a **pre-ingestion** step: human review. Leverage = reusing the existing "document → RAG" path.
L5	**Human approval before publish**	Review UI + single "approve" path into document creation	Mandatory human-in-the-loop ensures quality and control. The leverage is **placement**: the approval step sits between "AI draft" and "document in KB." One clear gate; no auto-publish. Builds trust and keeps the loop sustainable.

Summary: **L1 and L2** create the feedback loop (usage → gaps). **L3** makes the loop efficient (work on what matters). **L4 and L5** close the loop by turning gaps into new KB content through the existing doc pipeline and a single review step.

---

3. The problem (in systems terms)

- **Missing feedback loop:** The system has no path from "this question was not answered well" back to "add or fix content in the KB." So the KB is updated only when the franchisor proactively thinks of a topic or reacts to complaints.
- **Gaps are invisible:** The franchisor does not have a clear, prioritized view of "what our people are asking that we don't cover." So they don't know where to invest documentation effort.
- **Reactive documentation:** New or updated docs are created only after pain (repeated questions, tickets, or audits). There is no mechanism to **proactively** suggest "you should have a doc on X" with evidence (e.g. "asked 12 times by 8 locations").
- **Blank-page burden:** Even when the franchisor knows a topic is missing, they must write from scratch. The system does not propose a first draft, so the bottleneck remains "someone has to sit down and write."

In short: the **control loop** that would keep the KB aligned with actual demand is absent. The Knowledge Builder Agent is designed to close that loop.

---

4. Where the solution is targeted

The solution is **not** a replacement for the RAG pipeline or the document pipeline. It **inserts** three subsystems into the existing flow.

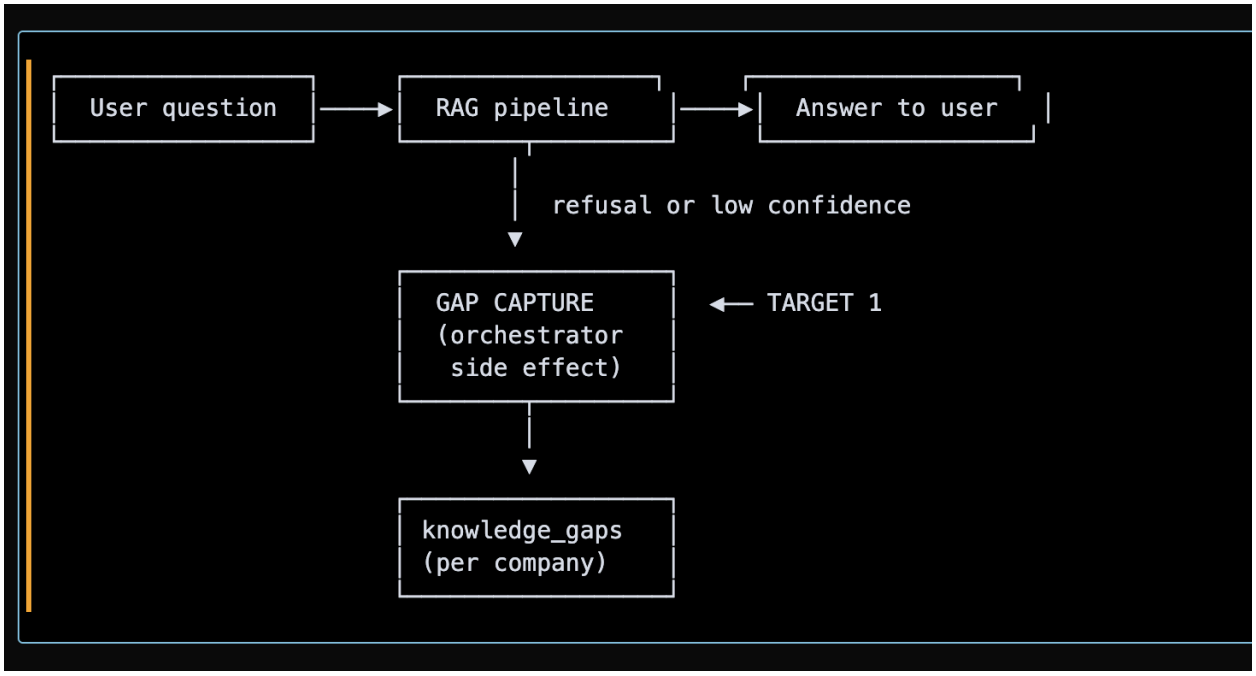
4.1 Gap detection (target: post-answer, plus tickets)

Location in system: Immediately after each AI chat completion, and at support ticket creation.

- **At chat completion:** After the RAG pipeline returns an answer, the orchestrator already has: the question, the retrieved chunks (or lack thereof), the model's response, and refusal/low-confidence detection. We add a **side effect**: if the response is a refusal or below a confidence threshold, write a structured **gap record** (or update an existing one via deduplication) in `knowledge_gaps`. Optionally correlate with a ticket if the user files one shortly after.
- **At ticket creation:** If the user had a recent chat session, link the ticket to the most recent low-confidence interaction and create/update a gap.

Leverage points used: L1 (signal), L2 (structured storage), L3 (implicit via occurrence count and later scoring).

Data flow: `User question → RAG → Answer → [existing] refusal/confidence logic → [new] gap capture → knowledge_gaps`. No change to how answers are generated; only an additional write after the fact.



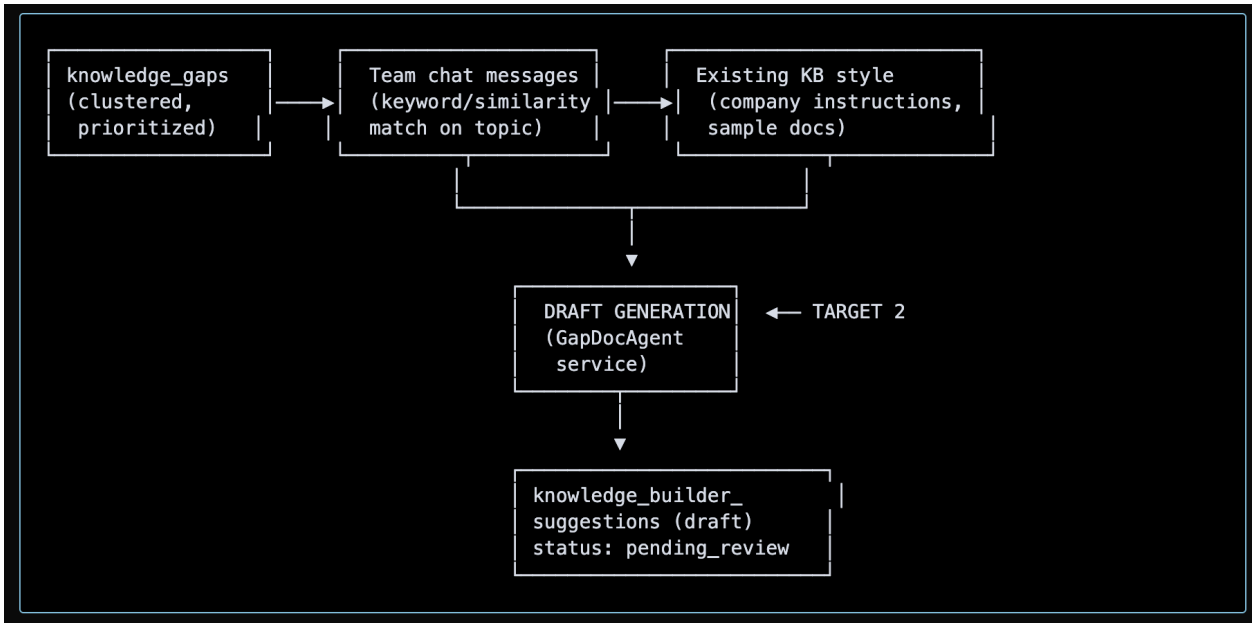
4.2 Draft generation (target: gap store → draft store)

Location in system: Offline or scheduled process that reads from `knowledge_gaps` (and optionally team chat) and writes to a **draft/suggestions** store.

- **Inputs:** Clustered gaps (same topic, e.g. 3+ occurrences), plus optional context from team chat messages that mention the topic.
- **Process:** An AI agent (separate from the chat LLM) takes the cluster + context + existing KB style and produces a **draft document** (title, body, suggested visibility). No direct interaction with the live RAG pipeline; this is a batch/scheduled job.
- **Output:** A row in a table such as `knowledge_builder_suggestions` with status `pending_review`. The draft is **not** in the KB yet; it is a candidate.

Leverage points used: L3 (only high-priority clusters get drafts), L4 (drafts are designed to become normal documents once approved).

Data flow: `knowledge_gaps` (open, clustered) + `channel_messages` (optional) → **GapDocAgentService** → `knowledge_builder_suggestions` (draft). The RAG pipeline and the document pipeline are unchanged at this stage; we only **create content** that will later be fed into the document pipeline.



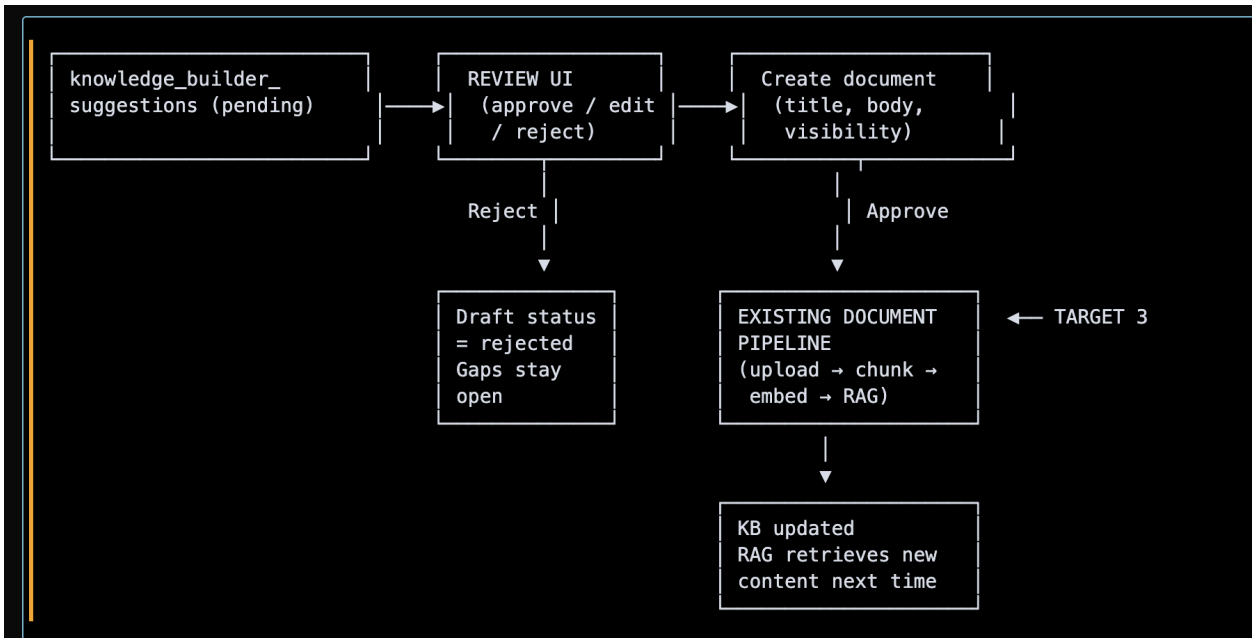
4.3 Review and publish (target: draft store → document pipeline)

Location in system: Between the draft store and the **existing document ingestion path**.

- **Review UI:** Franchisor admin sees pending drafts (in-app and/or via email). They can approve, edit then approve, or reject. Approval means “create a real document from this draft.”
- **Publish path:** On approve, the system creates a **document** (e.g. in `documents` with content from the draft) and triggers the **existing** document processing pipeline: store file or content, chunk, embed, write to `document_chunks`. From that point on, the new doc is part of the KB and the RAG pipeline will retrieve it like any other document. Optionally, source gaps are marked **resolved**.
- **Reject path:** Draft is marked rejected; gaps stay open. No change to the KB.

Leverage points used: L4 (reuse of document pipeline), L5 (single human gate before any write to the KB).

Data flow: `knowledge_builder_suggestions` (pending_review) → **Admin review** → Approve → **Create document** → [existing] **Document processing pipeline** (S3, worker, chunk, embed) → **RAG**. The Knowledge Builder does not implement its own ingestion; it **feeds** the existing pipeline.



---

5. End-to-end system view (with Knowledge Builder)

Putting it together, the system **with** the Knowledge Builder Agent looks like this:

1. **Franchisee asks a question** → Chat API.
2. **RAG pipeline** runs (retrieve chunks, LLM, answer). If the answer is weak or a refusal → **gap capture** writes/updates `knowledge_gaps`.
3. **Periodically (or on demand):** Gap clustering and prioritization run; for high-priority clusters, **draft generation** produces rows in `knowledge_builder_suggestions`.

4. **Franchisor admin** sees pending drafts in the review UI (or email); approves (with or without edit) or rejects.
5. **On approve:** A document is created and sent through the **existing document pipeline**; chunks and embeddings are created; the KB is updated.
6. **Next time** a franchisee asks a related question, the RAG pipeline **retrieves the new content** and can answer. The loop is closed.

The Knowledge Builder does **not** replace or duplicate:

- The RAG pipeline (retrieval, LLM, citations).
- The document processing pipeline (chunking, embedding, storage).
- The chat or ticket UX.

It **adds**:

- A **feedback path** from “bad or uncertain answers” to structured gaps (L1, L2, L3).
- A **draft path** from gaps (and optional team chat) to human-reviewable content (target 2).
- A **publish path** from approved draft into the existing document pipeline (L4, L5, target 3).

So the solution is targeted at **three specific insertion points** in an otherwise unchanged system: (1) post-answer gap capture, (2) gap-to-draft generation, and (3) draft-to-document handoff into the existing ingestion pipeline. That is the systems architecture of the Knowledge Builder Agent.